C# Nedir?

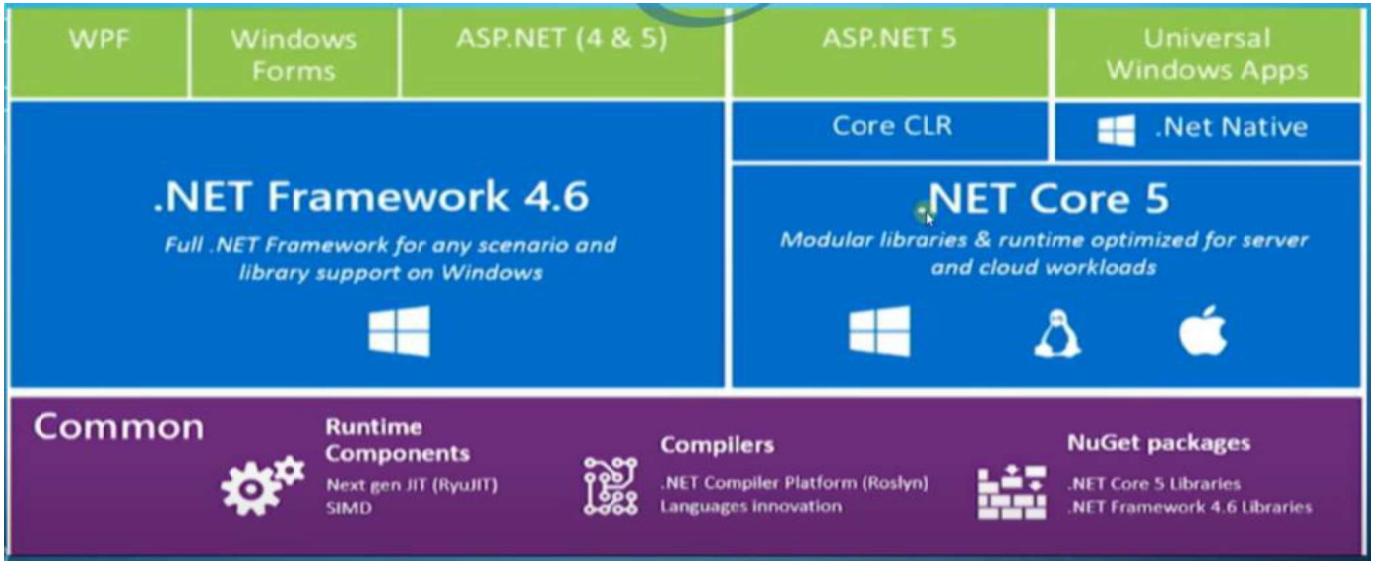
- Microsoft tarafından .NET çatısı altında geliştirilen ve gelişen modern programlama dilidir.
- Açık kaynak ve ücretsizdir.
- OOP(Nesne Yönelimli Programlama)'yi ve OOP'nin gelişimini destekler.
- Orta seviyeli bir dildir.
- Java'dan etkilenmiştir. Java'yı etkilemektedir.
- Derlenen bir programlama dilidir.

C# ile Neler Geliştirilebilir?

- Web Uygulamaları,
- Mobil,
- Web Servis,
- Servis Mimarileri,
- Konsol,
- DLL,
- Windows Form,
- Oyun,
- ERP, Muhasebe vs.

.NET Nedir?

- Microsoft'un developerlar için geliştirdiği teknolojileri sunduğu bir çatıdır.



Solution(Çözüm) ve Project(Proje)

- C# ile programlama yaparken Visual Studio'da proje ve çözüm yapıları ile karşılaşılabilir.
- Proje, içerisinde amaca dair kodsal çalışmaların yapıldığı bir bütündür.
- Solution ise içerisinde birden fazla projeyide barındırabilen daha geniş bir kavramdır.

.NET CLI Nedir?

- .NET komut satırı arayüzüdür.
- .NET uygulamaları geliştirmeyi, oluşturmayı, çalıştırmayı ve yayınlamayı sağlar.
- .NET SDK ile birlikte gelir.

Programlamanın Temel İlkeleri

- Kendini tekrar etme
- Anlamlı isimlendirme

Main Fonksiyonu Nedir?

- Main fonksiyonu uygulama ilk ayağa kalktığında(çalıştırıldığında) ilk tetiklenen(çalışan) fonksiyondur. Dolayısıyla ilk kodlar bu fonksiyonun içerisine yazılır.
- Uygulamalarda Program.cs dosyası başlangıç kodlarının bulunduğu yerdir. Csharp C# C++
- Başlangıç kodlarından kasıt uygulama ayağa kalktığında işletim sistemiyle iletişim kurabilecek metodun ve bu metot içerisinde başlangıca dair komutları barındıracak bir inşadır.

Main Fonksiyonu Nedir?

- Herhangi bir uygulama olsada bu main fonksiyonundan sade ve sadece 1 adet olmak zorundadır.
- Main fonksiyonu, uygulama da Program.cs dosyası içerisinde Main isminde bulunur.

Main Fonksiyonu Nedir?



Top Level Statements

- Sıradan bir işlem için bile oluşturulan console uygulamasında hiç yoktan basmakalıp kodların gelmesi gerekmektedir. Günlük hayatta basit kodlar inşa edebilmek ve test süreçlerinde hızlı denemeler yapabilmek için bu tarz bir kod bloğuna gerek yoktur.
- C# 9.0 ile gelen top level statements özelliği ile main fonksiyonunun zoraki imzasının tanımlanması kaldırılmıştır. Main fonksiyonu yazımı geliştiricinin kararına bağlıdır.

Top Level Statements

- C# 9.0’dan önce bu şekilde bir kullanım hata verecektir.
- Kurallar:
- 1. using blokları ile namespace arasındaki kodlar yazılabilir.
- 2. bu işlem sadece Program.cs dosyasında geçerlidir. Yani main fonksiyonunda yazılacak komutların direkt burada yazılmasına müsaade edilmekte lakin farklı bir dosyada bu işlemi gerçekleştiremeyiz.
- Genelde mikroservis mimarisinde kullanılır.

Açıklama/Yorum Satırı

- Açıklama satırları kodun niteliğini, anlaşılabilirliğini, kalitesini arttırabilmek için kullanılır.
- Krtirik noktalarda ve özet olarak kodları izah etmeliyiz.
- C#'ta
- 1. // tek satırlık yorum satırı oluşturmayı sağlar.
- 2. /**/ çok satırlık yorum satırı oluşturmayı sağlar.

Region

- Kod dosyasını kategorik hale getirmemizi sağlayan bir ön işlemci komutudur.
- Ön işlemci komutu editörün yorumlayabildiği bir yapılanmadır.
- Geliştiricinin yazmış olduğu kodu daha net görmesini ve kategorize etmesini sağlar.
- C#'ta region şu şekilde oluşturulur:
- #region matematiksel işlemleri yapan kısım
- #endregion

Todo(Yapılacaklar Listesi)

- Çok fazla bölümden oluşan projelerde bir kişiye verilen görevin nerede olduğunun bulunması zor olabilir. Yorum satırı kullanılsa dahi yüzlerce kod dosyası arasında bu yorum satırının bulunması zor bir hal alabilir. Bunun için todo yapısı kullanılır.
- C#'ta todo şu şekilde kullanılır:
- //todo 1 den 100 e kadar yazılacak.

Debugging

- Debugging hata ayıklama anlamına gelir ve programın hatalarını yok etmeye yönelik yazılan kodu gözden geçirme, inceleme, düzeltme aktiviteleridir.
- Breakkpoint ile kod belirli bir noktada bekletilir ve durum incelenebilir.
- C#'ta watch penceresi ile doğrudan izleme yapılabilir.
- Debug işlemi oldukça maliyetlidir ve uygulamanın çalışma hızını oldukça yavaşlatabilir. Bazı durumlarda uygulama debug edilmeden derlenip doğrudan çalıştırılabilir.

Değişken Nedir?

- Yazılımda işlenecek veriyi yazılım adına RAM'e yerleştirebilmek için değişkenleri kullanırız.
- C# programlama dilinde RAM'de veri tutabilmek/depolayabilmek için tanımlanacak olan değişkenin türü/veri türü bildirilmelidir.
- Bir değişkenle RAM'de alan tahsisinde bulunulduğunda buna Değer türlü değişken diyoruz. Yani tuttuğu değer bir normal değer olan değişkenlere değer türlü denmektedir.
- Değer türlü değişkenler sade ve sadece değeri tutan değişkenlerdir. Yani ad, soyad, doğum tarihi, yaş gibi değerleri tutabilirler.

Değişken Nedir?

Tür	Açıklama ve Bellek Alanı	Max-Min aralığı
bool	Doğru veya Yanlış (1 Bit)	0-1 (True - False)
char	Karakterler (16 Bit)	16Bit Unicode
sbyte	İşaretli Tam Sayı (8 Bit)	-128 ile 127 arası
byte	İşaretsiz Tam Sayı (8 Bit)	0 ile 255 arası
short	İşaretli Tam Sayı (16 Bit)	-32.768 ile 32.767 arası
ushort	İşaretsiz Tam Sayı (16 Bit)	0 ile 65.535 arası
int	İşaretli Tam Sayı (32 Bit)	-2.147.483.648 ile 2.147.483.647 arası
uint	İşaretsiz Tam Sayı (32 Bit)	0 ile 4.294.967.295 arası
long	İşaretli Tam Sayı (64 Bit)	-9.223.372.036.854.775.808 ile 9.223.372.036.854.775.807
ulong	İşaretsiz Tam Sayı (64 Bit)	0 ile 18.446.744.073.709.551.615 arası
float	Tek Kayan Sayı (32 Bit)	$\pm1,5*10^{-45}$ ile $\pm3,4*10^{38}$ arası
double	Çift Kayan Sayı (64 Bit)	$\pm5*10^{-324}$ ile $\pm1,7*10^{308}$ arası
decimal	Ondalıklı Sayı (128 Bit)	$\pm1,5*10^{-28}$ ile $\pm7,9*10^{28}$ arası

C# Temel Kuralları

- C# programlama dili büyük küçük harf duyarlılığına sahiptir.
- C# programlama dilinde tip güvenliği vardır.
- C#'ta değişken şu şekilde tanımlanır: degisken_turu degisken_adi;
- Değişkenlerin türü, adı ve değeri ayrıca metot isimleri(bellek adresleri) Stack'te tutulur.
- Kod çalışma zamanında RAM'de alan tahsisi gerçekleştirilir.
- Değişken adlarının belirli kurallara göre tanımlanması gerekir.
- Değişken isimlendirme kurallarıda vardır.
- Değişken isimlendirirken @ işareti kullanımı.

Değişken İsimlendirme Kuralları

- Genel olarak programlama dillerinde değişkenlere isim verirken uygulanan bazı yöntemler vardır. Bunlar: snake_case, PascalCase ve camelCase olarak adlandırılır.
- Örneğin;

```
int birinci_deger; // snake_case kullanımı
int BirinciDeger; // PascalCase kullanımı
int birinciDeger; // camelCase kullanımı
```

- Bu tarz değişken isimlendirmeleri zorunlu olmamakla birlikte bir düzen ve kod anlaşılabilirliği sağladığı için önerilmektedir.

@ operatörü kullanımı

- Değişken isimlerinde programatik keyword kullanılamaz! Eğer ki bir değişken isminde programatik olarak kullanılan bir keywordü vermek istiyorsanız eğer bunu @ operatörü ile kullanmanız gerekir.
- Örneğin int @static = 5;

Tanımlanmış Değişkene Değer Atama

- C# dilinde tanımlanmış bir değişkene doğrudan tanımlandığı satırda veya sadece değişken tanımlaması yapılarak, kodun geri kalan kısımlarında atama yapılabilir.
- Örneğin;

```
int birinci_deger = 5; // Değişkene tanımlandığı yerde atama
int birinci_deger;
birinci_değer = 15;
```

- Buradaki '=' operatörü matematiksel eşitlik anlamında değil atama operatörü olarak kullanılmıştır.

Tanımlanmış Değişkene Değer Atama

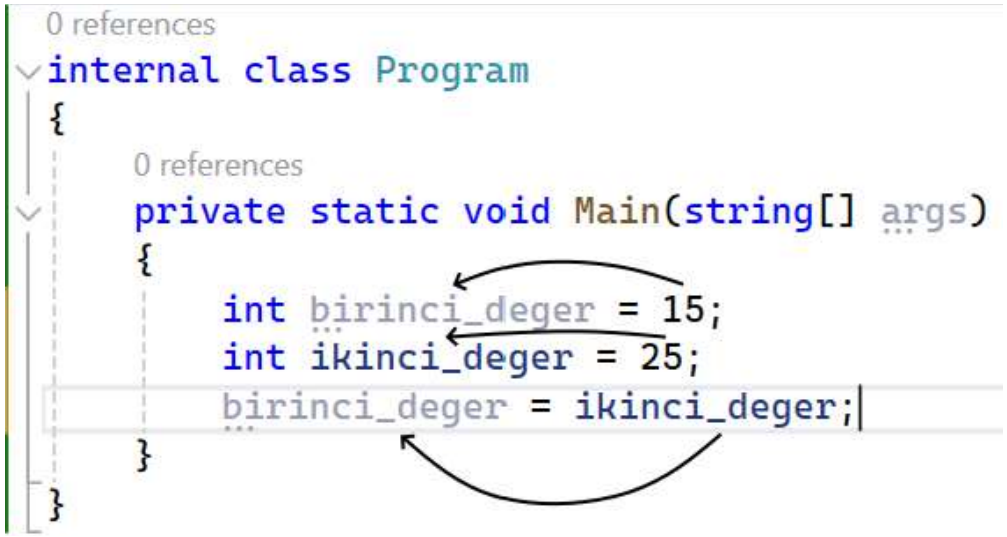
- Değişkene değer atanırken = işaretinin sol tarafına değişkenin kendisi gelirken sağ tarafına değeri gelecektir.

```
int birinci_deger = 15;
int ikinci_deger = 25;
int ucuncu_deger = 35;
İkinci_deger = ucuncu_deger;
```

- Bu durumda ikinci_deger kısmına değişkenin kendisi gelirken ucuncu_deger kısmına ucuncu_degerin kendisi değil değeri(35) gelecektir.

Tanımlanmış Değişkene Değer Atama

- Değişken tanımlaması ve ataması yapıldığında daima sağdaki değer soldaki değişkene atanır.



Kurallar

- Bir değişkene atanan en son değer geçerlidir.

```
int birinci_deger = 15;
birinci_deger = 25;
birinci_deger = 57;
```

- Bu durumda birinci_deger değişkenine son atanan değer 57'dir.
- Tanımlanmış olan değişkene türüne uygun bir değer atanmalıdır.

```
int birinci_deger = "selam";
```

- Bu durumda hata ile karşılaşılır. Çünkü int bir değişken tanımlanmış ama bu değişkene string bir değer atanmaya çalışılmaktadır.

Metinsel Değerler

- Temel olarak string değişken tipi ile ifade edilir.
- Çift tırnak içerisine değeri yazılır.
- Bir sayısal ifade metinsel olarak tutuluyorsa yazılım açısından metinsel olarak ifade edilir.
- Örneğin;

```
string isim = "Zafer";
string ifade = "Zafer 123";
string metinsel = "21";
string karisik = "Zafer123*!";
```

Karaktersel Değerler

- Temel olarak char değişken tipi ile ifade edilir.
- Tek tırnak içerisine değeri yazılır.
- Tek bir karakteri tutarlar.
- Örneğin;

```
char a_karakteri = 'a';
char yildiz_karakteri = '*';
char unlem_karakteri = '!';
char son_karakter = 'n';
```

Mantıksal Değerler

- Temel olarak bool değişken tipi ile ifade edilir.
- true veya false değer alabilir.
- Var-yok, doğru-yanlış, 1-0 gibi düşünülebilir.
- Atama yapılırken herhangi bir tırnak işareti kullanılmaz!
- Örneğin;

```
bool giris_yapildimi = true;
bool sart_saglandimi = false;
bool islem_tamamlandimi = true;
```

Sayısal Değerler

- Temel olarak int, float, double ve decimal gibi tipler ile ifade edilirler.
- Tamsayılarda varsayılan değer int, ondalıklı sayılarda doubledır.
- float değişken tanımlanırken değerin sonuna f veya F, double için d veya D, decimal için m veya M eklenir. Bunlara kapsam dışı atama yapılamaz!
- Örneğin;

```
int sayi1 = 15;
float sayi2 = 25.7f;
double sayi3 = 21.65D; // double sayi3 = 21.65;
decimal sayi4 = 2156.543m;
```

Tuple ile Değer Atama

- Tek bir syntax ile birden fazla değişkene değer atamak için kullanılır.
- Örneğin;

```
(int yas, string isim) tuple1 = (25, "Zafer");
(float boy, char karakter) tuple2;
tuple2 = (1.70f, 'z');
(int sayi1, string okul) tuple3;
tuple3.sayi1 = 30;
tuple3.okul = "PMYO";
```

Literals Düzenlemeler

- C# 7.0 ile gelen bir düzenleme ile büyük sayısal değerlerde ‘_’ ile basamak ayrımı yapılabilmesi sağlanmıştır.
- Örneğin;

```
int sayi1 = 1_000_000; // int sayi1 = 1000000;
```

Varsayılan Değer Atama

- Bir değişken class içerisinde tanımlanırsa varsayılan bir değere sahip olacaktır. Bunlar; string için null, char için \0, sayısal ifadelerde 0 ve bool için ise false olur.
- Main içerisindeki bir değişkene ise varsayılan değer doğrudan atanmaz! İlgili değerin elle verilmesi gerekir. Bir değişkene varsayılan değeri vermek için default sözcüğü kullanılır.

```
int sayi1 = default(int); // int sayi1 = default; c# 7.1 ile gelen özellik
bool giris_yapildimi = default(bool);
char karakter = default(char);
```

Tanımlanmış Değişkenin Değerini Okuma

- Bir değişkenin değerini elde edebilmek için değişkenin adından faydalanmaktayız. Bir değişkenin adı assign(=) operatörünün sağında yahut metotların parametrelerinde çağrılıyorsa ilgili değişkenin değeri gönderilir.
- Örneğin;

```
int sayi1 = 25;
Console.WriteLine(sayi1);
// 4 sayılı örnek
```

Değişkenlerin Faaliyet Alanları(Scope)

- Faaliyet alanı değişken ve fonksiyonların erişilebilirlik sınırlarını belirleyen alandır.
- Tanımlamalarda ve algoritmik çalışmalarda karışıklığı önleyen bir sınır çizer.
- C#'ta faaliyet alanları süslü parantezler ile belirlenir.
- Bir değişken sade ve sadece tanımlandığı scope içerisinden erişilebilir ve kullanılabilir.
- Süslü parantez ile istenilen yerde scope oluşturulabilir.

Değişmeyenler(Sabitler)

- Sabitler; değişmeyen değerleri tutmak için oluşturulmuş yapılardır.
- Süreçte var olan değeri değiştirilemez, değiştirilmeye çalışıldığı takdirde compiler tarafından hata verilir.
- C#'ta const anantar sözcüğü ile kullanılır ve açılımı constant'tır. Değişmeyendir. Prototip olarak değişkenlere çok benzer lakin davranışsal olarak değeri bir daha değiştirilemez. Özünde static yapılanmadır.
- Static yapılanma uygulama bazlı veri depolayabildiğimiz bellekte bir alandır. Static değişkenlerin const tan farkı değerlerinin değişebilmesidir.

Değişmeyenler(Sabitler)

- Bir diğer sabit türü readonly olarak düşünülebilir. Bu sadece okunabilir değişkenler tanımlamak için kullanılır.
- Const'tan farkı sadece tanımlandığı yerde değil, ayrıca constructor içerisinde de değeri atanabilir. Dependency Injection deseninde çok sık tercih edilir.
- Ayrıca static değildirler.
- Const'lar tanımlanır tanımlanmaz değerini almak zorundadır.
- Bir const tanımlandığında Stack'te ilgili türde bir alan tahsis edilecektir ve ilk atanan değer dışında bir daha değer kabul etmeyecektir.

Değişmeyenler(Sabitler)

- Const'lar değiştirilemez lakin istenildiği kadar okunabilir.
- Const değişkenlere tanımlama aşamasında değer verilmelidir.
- const degisken_tipi degisken_adi; şeklinde c#'ta tanımlanabilir.

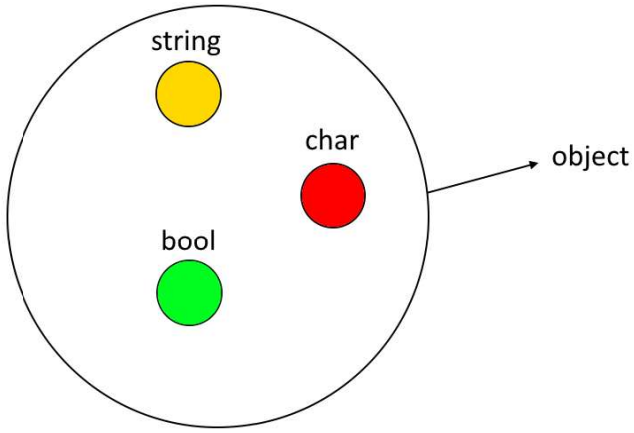
Değişken Atama Varyasyonları

- Aynı türden değişkenler tek bir satırda atanabilir. Örneğin

```
int sayi1 = 5;
int sayi2 = 10;
int sayi3 = 5, sayi4 = 10;
```

Object Türü

- Object türü tüm türleri karşılayabilen bir türdür. Boxing yöntemi ile değişken kutulanır ve bir object tür haline getirilir. Unboxing ile ise tekrar dışarı alınarak gerçek değer elde edilir.



var ifadesi

- Tutulacak değerin türüne uygun bir değişken tanımlayabilmek için kullanılan bir keywordtür.
- var keywordü kendisine atanan değerin türüne bürünür.
- Bir değişken türü değildir.
- Asıl amacı farklı diller arasında uyum sağlamaktaır.
- Örneğin;

```
var sayi1 = 15;
var isim = "Zafer";
var boy = 1.70f;
```

dynamic ifadesi

- var ile benzer bir kullanıma sahip olmakla birlikte var derleme aşamasında değerin türüne bürünürken, dynamic ise runtimeda(çalışma zamanında) değerin türüne bürünür.
- GetType() metodu ile değişkenlerin türü öğrenilebilir.
- Çalışma zamanında uzaktan gelen verileri karşılamak için kullanılır.

Tür Dönüşümleri(Type Conversion)

- Yazılım sürecinde elimizdeki değerlerin türlerini değiştirebilmekteyiz.
- Tür dönüşümleri elimizdeki verinin mahiyetindeki türe uygun işlemlere tabi tutulması için uygulanabilir.
- Örneğin elimizde string tipte "123" değeri varsa ve biz bunu 2 ile çarpmak istersek sorun yaşarız. Bu gibi durumlarda tür dönüşümü kullanılabilir.
- Ayrıca farklı servislerden gelen değerleri uygun türlere dönüştürmek içinde kullanılır.

Tür Dönüşümleri(Type Conversion)

- Tür dönüşümlerinde amaç türü dönüştürmektir. Yani elimizdeki veriye uygun bir türe geçiş yapmaktır. Elimizdeki veriyi uygun olmayan bir türe dönüştürmeye çalışırsak bu mümkün değildir. Hata verecektir. Amaç veriyi değiştirmek değil dönüştürmektir.
- Örneğin "Zafer" şeklindeki bir string ifadeyi inte çeviremeyiz.

Metinsel İfadelerin Diğer İfadelere Dönüştürülmesi

- Burada kullanılan 2 yöntem vardır. Bunlar Parse ve Convert fonksiyonlarıdır. Parse yöntemi sadece string dataları hedef türe dönüştürürken kullanılır.
- Parse kullanımı şu şekildedir: type.Parse(); burada type ile ifade edilen dönüştürülecek tiptir. int, bool vb.

```
string deger = "123";
int deger_int = int.Parse(deger);
```

Metinsel İfadelerin Diğer İfadelere Dönüştürülmesi

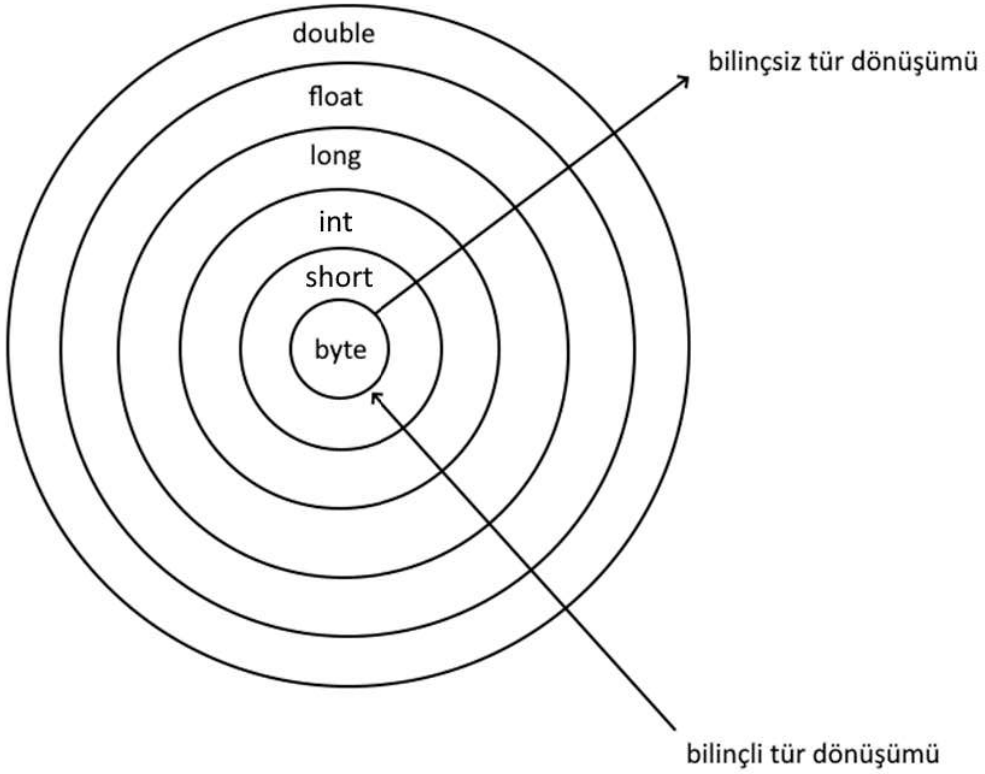
- Convert yöntemi sadece string için değil tüm tür dönüşümleri için kullanılabilmektedir; ancak yine tip uyumluluğu olmalıdır.

```
string giris_yapildimi = "TruE";
Convert.ToBoolean(giris_yapildimi);
```

Sayısal İfadelerin Kendi Arasında Dönüştürülmesi

- Sayısal ifadelerin dönüşümü bilinçli ve bilinçsiz tür dönüşümü olarak 2'ye ayrılır. Bilinçsiz tür dönüşümü kendinden daha geniş değerler tutabilen sayısal değişken tiplerine dönüşümde kullanılırken, bilinçli tür dönüşümü kendinden daha dar değerler tutabilen sayısal değişken tiplerine dönüşümde kullanılır.
- Bilinçli tür dönüşümünde veri kaybı olabilir.

Sayısal İfadelerin Kendi Arasında Dönüştürülmesi



Sayısal İfadelerin Kendi Arasında Dönüştürülmesi

```
byte sayi1 = 25;
int sayi2 = sayi1;

double sayi3 = 2500;
float sayi4 = (int)sayi3;
```

checked ve unchecked Yapıları

- Bilinçli tür dönüşümü sırasında veri kaybı olabilecek bir durum varsa bunu kontrol etmek için checked bloğu kullanılır ve veri kaybı olursa kullanıcıyı uyarır. unchecked bloğunda ise bu kontrol yapılmaz. Varsayılan durum unchecked'tir.

```
checked{
    double sayi1 = 5000d;
    byte sayi2 = (byte)sayi1;
    Console.WriteLine(sayi2);
}
```

Mantıksal Türlerin Sayısal Türlere Dönüşümü

- Mantıksal ifadeler sayısal türlere dönüştürülürken true değeri 1'e ve false değeri ise 0'a atanır.

```
bool deger = true;
int sayi = Convert.ToInt32(deger);
```

- Tam tersi durumda bir sayısal değer mantıksal değere dönüştürülecekse burada bir istisna vardır. 0 değeri false olurken 0 dışındaki tüm değerler true olarak dönüşecektir.

Sayısal Türlerin Mantıksal Türlere Dönüşümü

- Bir sayısal değer mantıksal değere dönüştürülecekse burada bir istisna vardır. 0 değeri false olurken 0 dışındaki tüm değerler true olarak dönüşecektir.

Karakter Türlerin Sayısal Türlere Dönüşümü

- Karakter(char) olarak tanımlanmış değişkenler sayısal ifadelere dönüştürülürken onların ascii tablo karşılıkları elde edilir.

```
char a_karakteri = 'a';
int b = Convert.ToInt32(a_karakteri);
```

Sayısal Türlerin Karakter Türlere Dönüşümü

- Karakter(char) olarak tanımlanmış değişkenler sayısal ifadelere dönüştürülürken onların ascii tablo karşılıkları elde edilir.

```
int sayi = 97;
char a_karakteri = Convert.ToChar(sayi);
```

Console.ReadLine() Kullanımı

- C# dilinde Console.ReadLine() ile kullanıcıdan değer alınabilir. Bu kod varsayılan olarak string tipte değer alır. Bu nedenle eğer aritmetiksel işlem yapılmak isteniyorsa; ilgili değerin uygun bir sayısal tipe(int, long vb.) çevrilmesi gerekir.

```
Console.WriteLine("Lütfen 1. sayiyi giriniz..:");
int sayi1 = Convert.ToInt32(Console.ReadLine());
Console.WriteLine(sayi1);
```

Aritmetik Operatörler

- C# ile aritmetiksel işlemleri yapabilmek için aşağıdaki operatörler kullanılır.

Operatör	İşlevi
+	Toplama
-	Çıkarma
*	Çarpma
/	Bölme
%	Mod Alma

Karşılaştırma Operatörleri

- C# ile karşılaştırma işlemleri yapabilmek için aşağıdaki operatörler kullanılır.

Operatör	İşlevi
<	Küçüktür
>	Büyüktür
<=	Küçük veya eşittir
>=	Büyük veya eşittir
==	Eşittir
!=	Eşit değildir

Mantıksal Operatörler

- C# ile mantıksal işlemleri yapabilmek için aşağıdaki operatörler kullanılır.

Operatör	İşlevi
&&	ve
||	veya
^	Ya da

İşlem Önceliği

- C#'ta işlem önceliği matematikte olduğu gibidir.

En Yüksek
()
!
*, /, %
+, -
<, >
= =, !=
&&
||
En Düşük

Artırma ve Azaltma Operatörleri

- C# ile artırma ve azaltma işlemlerini yapmak için ++ ve - - operatörleri kullanılır. Bunlar ilgili değeri 1 artırma veya 1 azaltma işlemine tabi tutarlar. Operatörün değişkenin solunda olması durumunda ilgili değer o satırda işlem görürken, operatör değişkenin sağında olursa ilgili değer bir sonraki satırda işlem görür.

Üzerine Ekleme Yığma Operatörleri

Operatör	Örnek Kullanım	Dengi
+=	İ += 5	İ = i+5
-=	İ -= 3	İ = i-3
*=	İ *= 4	İ = i*4
/=	İ /= 7	İ = i/7
%=	İ %= 2	İ = i%2

sizeof Operatörü

- C#'ta sizeof operatörü ile değişken tiplerinin kaç bytelık yer kapladığını integer olarak geriye döndürür. Doğrudan değişken adı ile çağrılamaz!

```
Console.WriteLine(sizeof(int));
Console.WriteLine(sizeof(long));
Console.WriteLine(sizeof(double));
```

typeof Operatörü

- Typeof operatörü verilen değerin tipini getirir. O tür ile ilgili bilgileri edinmek için kullanılan bir operatördür. Doğrudan değişken adı ile çağrılamaz!

```
string isim = "Zafer";
Type t = typeof(string);
Console.WriteLine(t);
```

is ve is null Operatörleri

- Boxing'e tabi tutulmuş bir değerin öz türünü öğrenebilmek için is operatörü kullanılır. True veya False olarak değer döndürür.

```
object sayi = 25;
Console.WriteLine(sayi is int);
```

- Değer tipli değişkenler (string hariç şu ana kadar gördüklerimiz) null değer alamaz. Null değer alabilen bir değişkenin değerinin null olup olmadığını kontrol etmek için is null operatörünü kullanırız.

```
string isim = null;
Console.WriteLine(isim is null);
```

as Operatörü

- Boxing edilmiş bir değeri dışarı çıkarmayı sağlar. () -> cast işleminden farkı eğer değer uymuyorsa hata vermek yerine null değer döndürür. Değer tipli değişkenler normal şartlarda null değer alamayacağı için buna uygun kullanılmalıdır.

```
object isim = 15;
string y = isim as string;
Console.WriteLine(y);
```

Değer tipli bir değişkeni null yapma

- Değer tipli bir değişkeni null yapabilmek için ? Kullanılır.

```
int? sayi = null;
```

?? Operatörü

- Elimizdeki değişkenin değerinin null olma durumuna istinaden farklı bir değeri göndermemizi sağlayan operatördür. Sol ve sağdaki türler aynı olmalıdır.

```
string isim = null;
Console.WriteLine(isim ?? "SERİN");
```

??= Operatörü

- Elimizdeki değişkenin değeri null ise yeni değeri değişkene atamayı sağlar. Sol ve sağdaki türler aynı olmalıdır.

```
string isim = null;
Console.WriteLine(isim ?? "SERİN");
Console.WriteLine(isim);

int? id = null;
id ??= 1;
```

Akış Kontrol Mekanizmaları

- Akış kontrol mekanizmaları, kodun akış sürecinde belirli şartlara göre farklı yönlendirmeleri yapmamızı ve farklı algoritmaları/kodları/yapılanmaları çalıştırmamızı sağlayan mekanizmalardır.
- if-else ve switch-case yapılanmaları olarak kullanılır. Bunlar arasında teknik olarak fark olmamasına karşın kullanım açısından bir fark yoktur.
- Algoritmalarda çok sık kullanılır ve oldukça önemli mekanizmalardır.

Switch-Case Yapısı

- Switch-case, kodun akışında belirli bir şarta göre yönlendirme yapmamızı sağlayan yapılanmadır.
- Sadece eşitlik durumu kontrol edilecekse o zaman switch kullanılabilir.

Switch-Case Yapısı

```
switch(kontrol_edilecek_deger)
{
        case deger1:
        // kontrol_edilecek_deger deger1'e eşitse yapılacak işlemler
        break;
        case deger2:
        // kontrol_edilecek_deger deger2'ye eşitse yapılacak işlemler
        break;
        default:
        // kontrol_edilecek_değer hiçbir degere eşit değilse yapılacak işlemlerdir. Default kullanımı zorunlu değildir!
        break;
}
```

Switch-Case Yapısı

- Yalnızca bir case durumu çalışacaktır. Bir şart sağlanırsa diğerlerine bakılmaz!
- Switch parantezinde kontrol_edilecek_deger bir değişken yahut statik bir değer olabilirken case bloklarındaki değerler kesinlikle statik olmak zorundadır. Caselerdeki değerler değişkenlerden alınamaz!
- Case bloklarının sıralaması ve defaultun yerleştirildiği yer önemli değildir.

Switch-Case Yapısı When Kullanımı

- Switch yapılanmasında eşitlik ile ilgili ek bir şart daha kontrol edilmek istenirse when yapısı kullanılır.

```
var sayi = 25;
switch (sayi)
{
    case 25 when (true):
        Console.WriteLine("Görüşürüz");
        break;
}
```

if Yapısı

- İf yapılanması, elimizdeki bir değerin eşitlik durumuda dahil tüm karşılaştırmaları yapmamızı sağlayan ve sonuca göre akışı yönlendirmemizi sağlayan bir yapılanmadır. Else kullanımı zorunlu değildir!

```
if(sart)
{

}
```

if-else Yapısı

- Bir şartın sağlandığı durumda bir kod bloğunu çalıştırmak için if yapısını kullanıyoruz. Eğer ilgili şart sağlanmaz ise çalışmasını istediğimiz bir kod bloğu var ise bunu da else yapısı ile belirtiyoruz.

```
if(sart){
// şart sağlanırsa yapılacaklar
}
else{
// şart sağlanmazsa yapılacaklar
}
```

if-else Yapısı

- Hem if bloğunda hem de else bloğunda çalışacak bir kod varsa bu kod mutlaka ilgili blokların dışına yazılmalıdır.(Kendimizi tekrar etmiyoruz!)
- if bloğu içerisinde tek satırlık bir işlem yapılacaksa süslü parantez kullanılmadan da işlem yapılabilir!(Scopesuz if yapısı)

if-else if-else Yapısı

- Sadece iki durumun değil farklı durumlarında kontrol edilmesi durumunda bu yapı kullanılabilir. Birden fazla şartı kontrol etmemizi sağlar. İf yapılanmalarından herhangi birisi sağlandıysa diğer ifler kontrol edilmeden geçilecektir.

```
if(ilk_sart){// ilk_sart sağlanırsa yapılacaklar}
else if(ikinci_sart){// ikinci_sart sağlanırsa yapılacaklar}
else if(ucuncu_sart){// ucuncu_sart sağlanırsa yapılacaklar}
else{// hiçbir şart sağlanmazsa yapılacaklar}
```

Type Pattern

- Object içerisindeki bir tipin belirlenmesinde kullanılan is operatörünün desenleştirilmiş halidir.
- is ile belirlenen türün direkt dönüşümünü sağlar.
- var anahtar sözcüğü ile kullanılabilir.

```
object sayi = 25;
if(sayi is int sayi_i)
{
    Console.WriteLine("Selam");
}
```

Constant Pattern

- Elimizdeki veriyi sabit bir değer ile karşılaştırabilmemizi sağlar.

```
object sayi = 25;
if(sayi is 25)
{
    Console.WriteLine("Selam");
}
```

Örnek1

- Girilen 3 sayı içerisinden en büyüğünü bulan kodu yazınız.

Örnek2

- Öğrencinin girdiği nota göre harf notunu hesaplayan kodu yazınız.

a) Başarı notu	Mutlak sistem karşılığı
AA.....	93,01-100,00
AB.....	83,67-93,00
BA.....	76,67-83,66
BB.....	69,67-76,66
BC.....	60,34-69,66
CB.....	56,61-60,33
CC.....	53,33-56,60
DC.....	49,00-53,32
DD.....	45,00-48,99
FF.....	.....44,99 ve aşağısı.

Döngüler

- Tekrar eden kodları bir koşula bağlı olarak belirli sayıda tekrarlayabilen yapılara döngü diyoruz.
- Koşul sağlandığı sürece ilgili döngü içindeki kodları tekrar eder.

for Döngüsü

- Aşağıda oluşturulan for döngüsü int i = 0; kodu ile bir i değişkeni tanımlamış ve buna 0 değerini atamıştır. Daha sonra bu değerin(0’ ın) 10’dan küçük olup olmadığını kontrol etmiştir. 10’dan küçük olduğu için ekrana Selam yazdırmış ve sonra i’yi 1 artırmıştır. Daha sonra tekrar bu değerin (1’in) 10’dan küçük olup olmadığını kontrol etmiş ve 10’dan küçük olduğu için tekrar ekrana Selam yazdırmıştır. Bu şekilde döngü devam eder.

```
for (int i = 0; i < 10; i++){Console.WriteLine("Selam");}
```

while Döngüsü

- Aşağıda oluşturulan while döngüsü bir önceki örnekteki for döngüsü ile aynı işi yapar. Burada i değeri daha önce tanımlanmıştır ve i değeri 10'dan küçük olduğu sürece while scopeu({}) içerisindeki işlemler yapılır. Döngü içinde i'nin artırıldığına(i++) dikkat edilmelidir. Yoksa sonsuz döngü meydana gelir.

```
int i = 0;
while(i < 10){
    Console.WriteLine("Selam");
    i++;}
```

do while Döngüsü

- do while döngüsü while'dan farklı olarak şart geçerli olmasa bile ilgili bloktaki kodu 1 kez çalıştırmayı sağlar.

```
int i = 0;
do
{
    Console.WriteLine("Selam");
    i++;
} while (i > 1000);
```

break ve continue anahtar sözcükleri

- Bu anahtar sözcükler döngüye müdahale etmemizi sağlar.
- break anahtar sözcüğü döngüyü sonlandırmayı sağlarken; continue anahtar sözcüğü yazıldığı yerdeki döngü iterasyonunu sonlandırıp bir sonraki iterasyondan devam edebilmeyi sağlar.

Örnek3

- 1’den 100’e kadar olan sayıları yazdırınız.

Örnek4

- 150 ile 450 arasındaki 15’in katı olan sayıları bularak ekrana yazdırınız.

Diziler(Arrays)

- Diziler içerisinde birden fazla aynı türde değer tutabilen yapılardır.
- Prosedürel programlamanın temel veri kümeleridir. Yani yazılımsal boyutta, yazılım adına RAM'de birden fazla değeri tek bir değişken altında bir veri kümesi halinde tutabilirler.
- Diziler veri kümeleri oldukları için, içlerindeki birden fazla veri üzerinde kümesel işlemler yapmamızı sağlayabilirler.
- Diziler, prosedürel programlamanın temel yapıları oldukları için daha gelişmiş yapılar olan koleksiyonlarında temelini oluştururlar ve gelişmelerine katkıda bulunurlar.

Diziler(Arrays)

- Diziler referans türlü değerlerdir. Yani nesnel yapılardır. Özlerinde classtırlar.
- Yapısal olarak RAM'de Heap'te tutulurlar.
- Dizi içerisine her türlü değer koyulabilir.(Değer türlü-referans türlü)
- Diziler içerisine koyulan değerler işlevsel olarak aynı mahiyette olmalıdır.
- Diziler içerisine eleman/değer eklendikçe bunları rastgele bir şekilde depolamaz! Düzenli bir şekilde sıralı depolayacaktır.
- Dizilerde eklenen elemanlar index(indis) ismini verdiğimiz numaralarla ardışık bir şekilde numaralandırılırlar.

Diziler(Arrays)

- İndex(indis) numarası her bir elemanı verilen ardışık sayıdır ve 0'dan başlayıp n-1'de sonlanır. n burada dizinin eleman sayısıdır.

type[] isim = new type[eleman_sayisi];

- Dizi tanımlama sürecinde eleman sayısı mecburi girilmek zorundadır. Yani dizide çalışacak değer adeti başta bildirilmelidir. Bu çok önemli bir sınırlılıktır.
- Bir dizi tanımlaması yapıldığı an bellekte o diziyi kullansakta kullanmasakta verilen eleman sayısı kadar alan tahsisinde bulunulur.

Diziler(Arrays)

- Kullanılmadığı halde diziler direkt olarak bellekten belirtilen eleman sayısı kadar alan tahsisinde bulunduğu için bu bir sınırlılıktır.
- Diziler alan tahsisi yapıldığında ilgili alanlara türüne uygun default(varsayılan) değerleri atarlar.

Diziler’de Değer Atama ve Değer Okuma

- Diziler’de değer atama ve okuma işlemleri index’ler üzerinden sağlanır.

```
int[] sayilar = new int[10];
sayilar[3] = 5;
Console.WriteLine(sayilar[3]);
```

- Diziler’de değer atama ve değer okuma işlemleri yapılırken dizinin sınırı aşılmamalıdır!
- Diziler’de değer ataması yapılırken sıralamayı göz önünde tutmak zorunda değiliz.

Diziler’de Değer Atama ve Değer Okuma

- Diziler’de eleman sayısı başlangıçta belirlendiği için duruma göre fazladan değer atayamayız. Bu bir sınırlılıktır.

For-each Iterasyonu

- C#'ta dizilerin her bir elemanına tek tek erişmek için for-each iterasyonu kullanılabilir.
- Örnek kullanım:

```
int[] elemanlar = { 11, 22, 33 };
foreach (var item in elemanlar)
{
    Console.WriteLine(item);
}
```

Metotlar

- Metotlar bir programın en küçük işlev birimi olarak düşünülebilir ve program içerisinde operasyonlar gerçekleştirmemizi sağlayan yapılardır.
- Metotlar sayesinde kod tekrarlarının önüne geçilebilir ve 1 kez yazılan metot pek çok yerde aynı işlevsellik ile kullanılabilir.
- Matematikteki fonksiyonlara oldukça benzemektedir.
- Aralarında ufak farklar olmasına karşın fonksiyon, metot ve yordam birbirine oldukça benzer yapılardır.

Genel Metot Tanımı

- Şu ana kadar gördüğümüz Main bir metottur.
- Metotlar classların veya structların içerisinde tanımlanabilir.
- Metot içinde metot tanımlanamaz(Local functionslar hariç!)
- Genel metot tanımı şu şekildedir:

```
[erişim belirleyicisi] [geri dönüş değeri] [metot adı](parametreler)//İmza
{
//Gövde
}
```

Yapılacak İşleme Göre Metotlar

- Yapılacak işleme(işlevine) göre 4 farklı türde metot oluşturulabilir. Bunlar:

1. Geriye Değer Döndürmeyen ve Parametre Almayan Metot
2. Geriye Değer Döndürmeyen ve Parametre Alan Metot
3. Geriye Değer Döndüren ve Parametre Almayan Metot
4. Geriye Değer Döndüren ve Parametre Alan Metot

Geriye Değer Döndürmeyen ve Parametre Almayan Metot

- Geriye değer döndürmeyen metotların geri dönüş değeri yoktur ve bunu belirtmek için ‘void’ anahtar sözcüğü kullanılır. Parametre almadığı için yaylı parantezlerin() arası boş bırakılır. Örneğin:

```
public void SelamYaz()
{
   Console.WriteLine("Selam");
}
```

Geriye Değer Döndürmeyen ve Parametre Alan Metot

- Geriye değer döndürmeyen metotların geri dönüş değeri yoktur ve bunu belirtmek için ‘void’ anahtar sözcüğü kullanılır. Parametre alan olduğu için yaylı parantezler arasında alınacak parametreler belirtilir. Örneğin:

```
public void Selamla(string isim, int yas)
{
    Console.WriteLine($"Merhaba {isim}, sen {yas} yaşındasın");
}
```

Geriye Değer Döndüren ve Parametre Almayan Metot

- Geriye değer döndüren metotların geri dönüş tipi metot imzasında belirtilmelidir. Metodun içerisinde de ‘return’ anahtar sözcüğü ile ilgili değer geriye döndürülmelidir. Parametre almadığı için yaylı parantezler arası boş bırakılır. Örneğin:

```
public int Topla()
{
    return 3 + 5;
}
```

Geriye Değer Döndüren ve Parametre Alan Metot

- Geriye değer döndüren metotların geri dönüş tipi metot imzasında belirtilmelidir. Metodun içerisinde de ‘return’ anahtar sözcüğü ile ilgili değer geriye döndürülmelidir. Parametre aldığı için yaylı parantezler arasına parametreler yazılmalıdır. Örneğin:

```
public int Carp(int sayi1, int sayi2)
{
    return sayi1 * sayi2;
}
```

Metotların Çağırılması

- Metotlar oluşturulduğu zaman kullanılacakları yerlerde çağırılmaları gerekmektedir. Değer döndüren metotlar tiplerine uygun değişkenlere atanarak istenildiği zaman kullanılabilir. Bazı metot çağırma örnekleri şu şekildedir:

```
SelamYaz();
Selamla("Zafer", 27);
int _topla = Topla();
int _carp = Carp(3, 4);
```